

Hybrid AI Research Assistant

A production-ready multi-tier web application integrating local Edge intelligence (SLMs) with commercial Cloud services (LLMs) for cost-optimal and high-performance cognitive computing.

Author: Ashish Singh

Repository: github.com/Ashishsingh2005-beep/Hybrid-AI-Research-Assistant

Date: July 2026

Frameworks: Streamlit, Llama.cpp, Google Gemini API

1. Executive Summary

The **Hybrid AI Research Assistant** is a software solution designed to solve the critical trade-off between operating costs, data privacy, and cognitive reasoning capabilities in large-scale AI applications. By utilizing a **Dual-Inference Engine (Hybrid Architecture)**, the application handles routine, low-complexity tasks locally using a **Small Language Model (SLM)** running directly on standard CPU hardware without any API costs. For complex reasoning, multi-document research, or long-form analysis, the system dynamically routes queries to a commercial **Cloud Large Language Model (LLM)** via API.

2. Technology Stack & Tools Used

The application leverages open-source local inference engines alongside cloud APIs to form a unified interface. Below is the list of core tools used in the development of this project:

Technology / Tool	Domain	Key Role in Project
Streamlit	Frontend UI Layer	Renders interactive dashboard, chat components, file upload, and custom styling.
Google Gemini API	Cloud LLM Engine	Performs complex reasoning, multi-document PDF analysis, and automated evaluation.
llama-cpp-python	Local SLM Loader	Loads quantized GGUF models directly into Python memory for low-latency offline inference.
llama-server.exe	Local C++ Server	Compiles and runs model server utilizing highly optimized C/C++ backend for Windows systems.
PyPDF (pypdf)	PDF Parser	Extracts textual contents from scientific research papers and reports for document QA.
Plotly & Pandas	Analytics Dashboard	Processes inference benchmarks (tokens/sec, cost, latency) and renders comparison bar charts.
psutil	System Diagnostics	Monitors background execution states and manages active local llama servers.

3. Core Application Features

A. Intelligent Routing (Auto-Routing Mode)

A custom decision-making router evaluates every user query before execution based on rules:

- *Word Count Threshold:* Short questions (e.g., < 50 words) are handled locally by the SLM to reduce cloud API billing.
- *Capability Detection:* Queries requiring real-time/latest data (e.g., 'news', 'weather') or uploaded PDF context are automatically escalated to the Cloud LLM.

B. Offline Capability & Simulated Mode

- The local engine works entirely offline without internet, preventing sensitive data leakage.
- Built-in *Simulated Offline Mode* automatically intercepts file-missing errors and provides lightweight mocks to allow design prototyping and UI evaluation on systems without model files downloaded.

C. Side-by-Side Model Comparison

- Developers can run parallel inference on Local SLM vs. Cloud LLM for the same prompt.
- Outputs live metrics (latency, speed in tokens/second, and exact API costs).

D. Automated AI Critique (Auto-Evaluation)

- The Cloud LLM acts as an independent judge on the comparison screen, evaluating both generated answers for logical consistency, accuracy, and grammar, outputting a professional scorecard.

4. Key Architectural Flow

Below is a conceptual representation of the query routing workflow in the hybrid assistant:

